



UNIVERSITY OF
CAMBRIDGE

Department of Computer
Science and Technology

May 2026

Query-based Dependent Type Elaborator

Submitted in partial fulfilment of the requirements for the
Computer Science Tripos, Part II

Declaration

I, *Author of College*, being a candidate for the Computer Science Tripos, Part II, hereby declare that this dissertation and the work described in it are my own work, unaided except as may be specified below, and that the dissertation does not contain material that has already been used to any substantial extent for a comparable purpose.

Signed:

Date: May 2026

Proforma

Candidate Number	TODO
Title	Query-based Dependent Type Elaborator
Examination	Computer Science Tripos — Part II, 2026
Word Count	8,532 ¹
Code Line Count	15,774 ²
Project Originator	The candidate
Project Supervisor	Dr Jon Sterling

0.1 Original aims of the project

Dependently typed languages — such as Lean, Agda, and Rocq — require type checkers that execute arbitrary programs at compile time. Most existing implementations operate in batch mode, re-checking entire files after each edit. This project set out to investigate whether a query-based, incremental approach could reduce the latency of elaboration in an interactive setting, by applying the “Build Systems à la Carte” framework to dependent type elaboration.

0.2 Work completed

TODO: fill in once evaluation is complete.

0.3 Special difficulties

None.

¹Computed via `pdftotext` on the main body (Introduction through Conclusion), including code blocks, tables, and captions.

²Computed with `cloc`.

Abstract

Dependently typed languages require expensive type checking, yet most elaborators operate in batch mode, re-checking entire files after each edit. This project implements an incremental, query-based elaborator for a dependently typed language supporting dependent function types, universe polymorphism, and inductive types with recursors. The elaborator is built on a formalisation of the “Build Systems à la Carte” framework in Lean 4, using a Shake-based incremental build system to memoise elaboration queries across edits. We evaluate the system on a standard library and conversion checking benchmarks, comparing incremental re-elaboration against batch re-elaboration.

Acknowledgements

TODO

Contents

Proforma	3
0.1 Original aims of the project	3
0.2 Work completed	3
0.3 Special difficulties	3
1 Introduction	8
1.1 Motivation	8
1.2 Prior art	8
1.3 This project	9
1.4 Contributions	9
1.5 Headline result	9
1.6 Overview	9
2 Preparation	11
2.1 Dependent type theory	11
2.1.1 Syntax	11
2.1.2 Judgement forms	11
2.1.3 Typing rules	12
2.1.4 Definitional equality	12
2.1.5 Universe polymorphism	12
2.1.6 Inductive types	13
2.2 Elaboration	13
2.2.1 Bidirectional type checking	13
2.2.2 Normalisation by evaluation	14
2.2.3 Conversion checking	14
2.3 Incremental computation	14
2.3.1 Build systems à la carte	14
2.4 Related work	15
2.4.1 Efficient elaboration	15
2.4.2 Lean 4	15
2.4.3 Self-adjusting computation and Adapton	15
2.4.4 Salsa	15
2.4.5 Sixty	15
2.5 Development methodology	16
2.5.1 Version control and testing	16
2.5.2 Tooling	16
2.6 Starting point	16
3 Implementation	17
3.1 Architecture	18
3.2 Parsing	19

3.2.1	Green trees	19
3.2.2	Language server integration	21
3.2.3	Error recovery	21
3.3	Bidirectional type checking	21
3.4	Normalisation by evaluation	22
3.4.1	Values and variables	23
3.4.2	Closures	23
3.4.3	Evaluation	23
3.4.4	Weak head normalisation	23
3.4.5	Quotation	24
3.5	Conversion checking	24
3.5.1	Eta-conversion	25
3.6	Inductive types	25
3.7	Incremental elaboration	26
3.7.1	Query wiring	26
3.7.2	The elaboration monad	27
3.7.3	Granularity of queries	27
3.7.4	Dependently-typed query results	27
3.7.5	Query dependency graph	27
3.7.6	Glued evaluation and dependency tracking	28
3.7.7	The Shake algorithm	28
3.7.8	Early cutoff	29
3.7.9	Native build system implementations	29
3.8	Language server	30
3.8.1	Incremental test harness	30
3.9	Repository overview	31
3.10	Formal verification	31
3.10.1	Refinements over BSALC	31
3.10.2	Correctness	32
3.10.3	The free theorem	33
3.10.4	LessBusy	33
3.10.5	Shake	34
4	Evaluation	36
4.1	Correctness	36
4.2	Batch elaboration performance	36
4.3	Conversion checking performance	36
4.4	Scaling benchmarks	37
4.5	Incremental re-elaboration	38
4.6	Discussion	39
5	Conclusion	40
5.1	Summary	40
5.2	Returning to the claims	40
5.3	Extensions beyond the proposal	40
5.4	Lessons learnt	41
5.5	Future work	41
	Bibliography	42
	Appendices	44
A	Project Proposal	45

Chapter 1

Introduction

1.1 Motivation

In a conventional compiler, changing a function body without changing its type cannot affect downstream type checking. In a dependently typed language, types can mention terms, so type checking may need to reduce through definition *bodies* during conversion checking. When I edit a single definition in a file in Lean [1], the elaborator re-checks every subsequent definition, even if most of them never unfold what I changed.

Lean [1], Agda [2], and Rocq [3] all treat a file as a sequence. Across files, each caches compiled results per file. Within a file, Lean saves snapshots of elaboration state at each top-level command and resumes from the last valid one; Agda caches results per declaration for the unchanged prefix; Rocq re-checks from the first modified sentence onward. An edit in the middle invalidates the entire suffix.

If a definition's body changes but its type is unaffected, definitions that depend only on its type need not be re-checked. But the dependency structure is discovered dynamically during elaboration: conversion checking may or may not unfold a given definition, depending on the specific terms being compared. Tracking which results depend on which, and recomputing only what has been invalidated, requires a build system that supports dynamic dependencies.

1.2 Prior art

Query-based incrementality has been applied to conventional compilers: rust-analyzer [4] uses the Salsa framework [5] for incremental type checking of Rust. In these languages, the compilation boundary is the type signature; changing a function body without changing its type cannot affect downstream checking. Fredriksson's Sixty [6] applies query-based incrementality to a dependently typed language, but does not formalise the underlying build system or verify that incremental results agree with batch elaboration. No mainstream proof assistant uses query-based incrementality.

Mokhov, Mitchell, and Peyton Jones [7] show that build systems can be characterised by a single polymorphic type, the *Task*, parameterised by the class of effects a task is allowed to perform. *Shake* supports dynamic dependencies (discovered during execution rather than declared up front) and *early cutoff* (skipping dependents when a recomputed result is unchanged). The

elaborator discovers which definitions it must unfold only as it runs, and early cutoff prevents unnecessary re-elaboration when an intermediate result has not changed. These are the properties we need.

1.3 This project

I apply that framework to elaboration. The elaborator decomposes its work into queries: “what is the type of this constant?”, “what are the errors in this definition?”, “what is the parsed AST of this file?”, each memoised by a build system. When a definition changes, only the queries that transitively depend on it are reconsidered; if a recomputed result is unchanged, its dependents are not recomputed.

The elaborator supports dependent function types, Tarski-style universes with universe polymorphism, and inductive types with recursors, enough to elaborate a standard library of arithmetic, equality, well-founded recursion, and algebraic structures. The implementation is in Lean 4, and the build system framework is formalised in the same language: three build systems (batch, memoising, and incremental) are proven to agree with batch evaluation.

1.4 Contributions

- A formalisation of the “Build Systems à la Carte” framework in Lean 4, refined with dependent query types ($\text{Val} : \text{Key} \rightarrow \text{Type}$), a separation of inputs from queries, and a well-foundedness condition on the dependency relation. Three build systems (Busy, LessBusy, Shake) are proved correct: their results equal batch evaluation by construction.
- An incremental elaborator for a dependently typed language supporting dependent function types, a hierarchy of Tarski-style universes with universe polymorphism, inductive types with recursors, and structures with projections. Elaboration is decomposed into per-declaration queries executed by the formalised build system.
- An evaluation on a standard library and synthetic benchmarks, comparing incremental re-elaboration against batch elaboration and against Lean 4.

1.5 Headline result

The incremental elaborator is tested on a standard library of 2,300 lines across 10 files, covering arithmetic, equality, well-founded recursion, and algebraic structures. After changing the body of `Nat.add` in `Nat.qdt` (a file imported by five others), the incremental rebuild re-elaborates only the queries whose dependencies changed. The remaining queries are verified by fingerprint comparison and reused. Measurements are in Section 4.5.

The build system executing this rebuild is one of three inhabitants of a verified `Build` type, proven in Lean 4 to produce the same results as batch evaluation. The proof, the elaborator, and the standard library share the same language, types, and definitions.

1.6 Overview

Section 2 introduces background: the build system framework of Mokhov et al. and our refinements, the dependent type theory we implement, bidirectional type checking, normalisation by evaluation, and related work. Section 3 describes the elaborator’s architecture and each pipeline

component: parsing, type checking, evaluation, conversion checking, inductive type elaboration, and incremental build and language server infrastructure. Section 4 presents measurements of correctness, batch and incremental performance, and conversion checking speed. Section 5 summarises the results and discusses future work.

Chapter 2

Preparation

2.1 Dependent type theory

We implement a dependent type theory in the tradition of Martin-Löf [8]. We assume familiarity with the simply typed lambda calculus and focus on what distinguishes our system.

2.1.1 Syntax

The core theory maintains a syntactic distinction between *types* and *terms* (Tarski-style universes). The grammar is:

$$\begin{aligned} \ell &::= n \mid 0 \mid \text{succ}(\ell) \mid \max(\ell, \ell) && \text{(universe levels)} \\ A, B &::= \text{Type}_\ell \mid \Pi(x : A).B \mid \text{El}(t) && \text{(types)} \\ t, s &::= \text{Type}'_\ell \mid x_i \mid c.\{\bar{\ell}\} \mid \lambda(x : A).t \mid ts \mid \Pi'(x : t).s \mid t.k \mid \text{let } x : A := t; s && \text{(terms)} \end{aligned}$$

Universe levels ℓ form an algebra of named variables n , zero, successor, and binary maximum. Types include universes Type_ℓ , dependent function types $\Pi(x : A).B$, and the decoding operation $\text{El}(t)$ converting a term-level code into a type. The term formers Type'_ℓ and $\Pi'(x : t).s$ are codes for the corresponding type formers. Variables x_i are de Bruijn indices, $c.\{\bar{\ell}\}$ are global constants applied to universe level arguments, and $t.k$ projects the k -th field.

For example, the type $\text{Nat} \rightarrow \text{Nat}$ is $\Pi(x : \text{El}(\text{Nat})). \text{El}(\text{Nat})$, where Nat is a term-level constant of type Type_0 . El decodes the term Nat into the type it represents. Without it, Nat cannot appear where a type is expected — the syntactic categories are disjoint.

In Russell-style theories (as in Lean 4), types and terms share a single syntactic sort: Nat can appear directly as a type without coercion. In our Tarski-style formulation, every position in the syntax tree is unambiguously either a type or a term. We prefer Tarski style because the syntactic separation simplifies the metatheory of our system.

2.1.2 Judgement forms

The theory has six judgement forms, parameterised by a global environment Δ and a local context Γ :

$\vdash \Delta \text{ sig}$	global environment well-formedness
$\Delta; \Gamma \vdash$	context well-formedness
$\Delta; \Gamma \vdash A \text{ type}$	type well-formedness
$\Delta; \Gamma \vdash t : A$	term typing
$\Delta; \Gamma \vdash A \equiv B \text{ type}$	judgemental type equality
$\Delta; \Gamma \vdash a \equiv b : A$	judgemental term equality

Δ maps constant names to their definitions and types. Γ is a telescope of typed bindings.

2.1.3 Typing rules

Type formation follows the grammar: Type_ℓ is well-formed in any well-formed context, $\text{El}(t)$ when t has type Type_ℓ , and $\Pi(x : A).B$ when A and B are.

The two central typing rules are introduction and elimination of dependent functions:

$$\frac{\Delta; \Gamma \vdash A \text{ type} \quad \Delta; \Gamma, x : A \vdash b : B}{\Delta; \Gamma \vdash \lambda(x : A).b : \Pi(x : A).B} \Pi\text{-I} \quad \frac{\Delta; \Gamma \vdash f : \Pi(x : A).B \quad \Delta; \Gamma \vdash a : A}{\Delta; \Gamma \vdash f a : B[a/x]} \Pi\text{-E}$$

Lambdas introduce Π types; application eliminates them, substituting the argument into the codomain. The remaining rules are standard: variables by context lookup, constants by environment lookup, let-bindings by checking the bound value and then the body, and the conversion rule allowing a term of type A to be given type B when A and B are definitionally equal.

2.1.4 Definitional equality

Definitional equality is the least congruence generated by:

- β -reduction: $(\lambda(x : A).b) a \equiv b[a/x]$
- δ -reduction (definition unfolding): $c.\{\bar{\ell}\} \equiv t$ when $\Delta(c) = t$
- ζ -reduction (let inlining): $\text{let } x : A := a \text{ in } b \equiv b[a/x]$
- η -conversion: $f \equiv \lambda(x : A).f x$ at function type
- ι -reduction (inductive type computation rule): recursor applied to a constructor computes to the corresponding branch

The full relation includes congruence, symmetry, transitivity, and closure under conversion for types.

2.1.5 Universe polymorphism

Universes are non-cumulative: Type_u does not belong to Type_v for $v > u$. Definitions may be parameterised by universe level variables:

```
def id.{u} (α : Type u) (x : α) : α := x
```

At each use site, levels are instantiated explicitly: $\text{id}.\{0\}$ or $\text{id}.\{v\}$. There is no universe level inference.

2.1.6 Inductive types

The theory supports user-defined inductive families. An inductive declaration introduces a type, its constructors, and a *recursor*. We illustrate with `Vector`:

```
inductive Vector.{u} (α : Type u) : Nat → Type u where
  | nil : Vector α Nat.zero
  | cons (n : Nat) (head : α) (tail : Vector α n) : Vector α (Nat.succ n)
```

It generates:

- The type `Vector.{u} : (α : Type u) → Nat → Type u`
- Constructors `Vector.nil.{u} : (α : Type u) → Vector α Nat.zero` and `Vector.cons.{u} : (α : Type u) → (n : Nat) → α → Vector α n → Vector α (Nat.succ n)`
- A recursor `Vector.rec.{v, u}` that eliminates a `Vector` into a motive type, which is dependent on the thing being eliminated, by providing a case for `nil` and a case for `cons`:

```
Vector.rec.{v, u} :
  (α : Type u) →
  (C : (n : Nat) → Vector α n → Type v) →
  C Nat.zero (Vector.nil α) →
  ((n : Nat) → (head : α) → (tail : Vector α n) → C n tail → C (Nat.succ n)
  (Vector.cons α n head tail)) →
  (n : Nat) → (xs : Vector α n) → C n xs
```

All computation on inductives proceeds through the recursor; there is no primitive pattern matching. The computational rule (*iota-reduction*) fires when the recursor is applied to a constructor — applying `Vector.rec` to `Vector.cons` reduces to the `cons` branch with the fields and recursive result as arguments.

As an example, `Vector.map` applies a function to each element:

```
def Vector.map.{u, v} (α : Type u) (β : Type v) (n : Nat) (f : α → β)
  : Vector.{u} α n → Vector.{v} β n :=
  Vector.rec.{v, u} α (fun k _ => Vector.{v} β k)
    (Vector.nil.{v} β)
    (fun m h _ => Vector.cons.{v} β m (f h))
  n
```

Structures (single-constructor inductives) support *projections* `t.k`, extracting the k -th field from a constructor application.

2.2 Elaboration

The typing rules above are declarative: they specify *what* is well-typed, not *how* to check it. Elaboration is the algorithmic counterpart.

2.2.1 Bidirectional type checking

Bidirectional type checking (Dunfield and Krishnaswami [9]) splits the typing judgement into two modes: *checking*, where the expected type is known and pushed into the term, and *inference*, where the type is synthesised bottom-up. The algorithm is syntax-directed and eliminates backtracking. A subsumption rule bridges the two modes via conversion checking.

I omit metavariables and unification, as their complexity is orthogonal to incrementality. The programmer must provide all type information explicitly.

2.2.2 Normalisation by evaluation

Type checking requires deciding definitional equality, which requires reducing terms. *Normalisation by evaluation* (NbE; Abel [10]) evaluates syntax into a semantic domain of values, where equality is checked by structural comparison rather than repeated rewriting. NbE performs substitution in bulk through environments rather than one variable at a time, and stops at weak head normal form. The implementation is described in Section 3.4.

2.2.3 Conversion checking

Glued evaluation ([11]) pairs each defined constant with a folded form (the constant itself) and an unfolded form (its definition body, available lazily), so that conversion checking can compare folded forms first and only unfold when heads differ. Combined with a three-mode approximate algorithm (rigid, flex, full), this avoids unnecessary reduction and, for incrementality, avoids creating unnecessary dependencies. The full algorithm is described in Section 3.5.

2.3 Incremental computation

The dominant cost in elaboration is conversion checking. Type checking a single definition may trigger many conversion checks, each of which may unfold and normalise arbitrarily large terms. In an interactive setting, most of these checks need not be repeated — only those whose dependencies actually changed. A query-based system can track which definitions each check unfolded, and skip the rest.

2.3.1 Build systems à la carte

Mokhov, Mitchell, and Peyton Jones [7] observe that build systems bring outputs up to date with respect to changed inputs. They capture Make, Shake, Bazel, and others as instances of a single polymorphic type.

The central abstraction is the *task*: a recipe that requests the values of other keys through a callback, with the build system deciding how those requests are fulfilled. A task is polymorphic in an effect — it works with any monad the build system chooses. The constraint on the monad determines what the task can do: **Applicative** means all fetches are declared upfront (static dependencies); **Monad** lets the task inspect a result and decide what to fetch next (dynamic dependencies).

The paper decomposes build systems along two axes: the *scheduler* determines execution order, while the *rebuilder* determines whether a key needs recomputation:

	Topological	Restarting	Suspending
Dirty bit	Make	Excel	-
Verifying traces	Ninja	-	Shake
Constructive traces	CloudBuild	Bazel	-
Deep constr. traces	Buck	-	Nix

Dependent type elaboration needs dynamic dependencies (the elaborator discovers which definitions it must unfold only as it runs) and early cutoff (skipping dependents when a recomputed result is unchanged). This places us at the Shake cell.

The polymorphism of the task type makes correctness modular: the elaborator defines tasks without knowing which build system executes them, and vice versa. The formalisation of this framework in Lean 4, including correctness proofs for three build systems, is presented in Section 4.

2.4 Related work

2.4.1 Efficient elaboration

Kovacs’s *smalltt* [11] combines higher-order abstract syntax, glued evaluation, and approximate conversion checking, achieving type-checking speeds that outperform production systems. I adopted approximate conversion checking and glued evaluation from it.

2.4.2 Lean 4

Lean 4 [1] is the primary reference for surface language and inductive types. Our elaborator supports the same declaration forms (`def`, `inductive`, `structure`, `axiom`) and recursor-based elimination. The core theory departs from Lean’s kernel in using Tarski-style universes and omitting metavariables and implicit arguments. Lean 4’s `Prop` is impredicative and `Type u` is predicative; our system has only predicative universes.

2.4.3 Self-adjusting computation and Adapton

The theoretical ancestor of fine-grained incremental computation is *self-adjusting computation* [12], which tracks data dependencies at runtime and propagates changes through a dependency graph. Adapton [13] refines this with demand-driven evaluation: computations are re-executed only when their results are demanded. The “Build Systems à la Carte” framework [7] applies these ideas at the granularity of named build targets rather than individual memory cells; our formulation inherits this coarser granularity.

2.4.4 Salsa

Salsa [5] is a Rust framework for on-demand, incrementalised computation, used by rust-analyzer [4] for incremental type checking of Rust. It implements query-based incrementality with memoisation and early cutoff, similar to the Shake cell. I target a dependently typed language, where conversion checking blurs the boundary between signatures and bodies, and provide a formal correctness proof.

2.4.5 Sixty

Fredriksson’s *sixty* [6] applies query-based incrementality to a dependently typed language, using a Haskell library inspired by BSALC. I formalise the build system in Lean 4 with a machine-checked correctness proof, and implement the elaborator in the same language, so the two share types and definitions.

2.5 Development methodology

The elaborator was built in three phases: a batch elaborator for the core theory (Pi types, universes, bidirectional checking), then inductive types and structures, then the incremental query architecture. Each phase was validated against the example code, including a small “standard library” before moving to the next.

2.5.1 Version control and testing

The codebase is hosted on GitHub. The dissertation is a separate Typst repository, also on GitHub. Commits to the elaborator are checked by `lake build` in CI; the `stdlib` and test suites run as part of the build.

The primary correctness test is the standard library: 2,300 lines across 39 files, re-elaborated from scratch on every run. The `stdlib` includes well-founded recursion, the Ackermann function with its reduction lemmas, proof irrelevance of accessibility predicates, and an algebraic hierarchy of semigroups through commutative groups. Elaborating these exercises deep chains of *iota*-reduction through nested recursors, universe polymorphism at every step, and the full pipeline from parsing through conversion checking. A test harness (`Qdt/Lsp/Test.lean`) simulates editor interactions, setting file contents, triggering rebuilds, and asserting on diagnostics and hovers. It covers pathological incremental scenarios: swapping definitions so forward references become backward, renaming and checking that dependents report unbound variables, moving a definition between files, and cyclic imports. Additional suites include Church-encoded normalisation benchmarks.

The build system’s correctness proofs (`Busy`, `LessBusy`, `Shake`) replace testing of the incremental layer: any inhabitant of `Build` is correct by construction.

2.5.2 Tooling

Development used VSCode with the Lean 4 extension, Lake as the build system, and a C FFI for the performance-critical `Shake` implementation. Lean’s macro system embeds `qdt` programs directly in Lean source files as inline test cases, since `qdt`’s syntax is a subset of Lean’s.

2.6 Starting point

I had experience implementing type checkers for simply typed languages, but had not built a dependently typed elaborator or a build system. I was familiar with type theory and had used Lean 4 as a proof assistant, but not as a general-purpose programming language for a project of this scale.

No code was written before the project started. The project was initially planned in Rust, but I switched early to OCaml, then to Lean 4. No existing codebases were used as a basis; I consulted `smalltt`, `sixty`, and Lean 4’s source as references.

Chapter 3

Implementation

This chapter describes the implementation of the elaborator. We follow the pipeline from source text to elaborated output: parsing, bidirectional type checking, normalisation by evaluation, conversion checking, inductive type elaboration, and the incremental build architecture that ties them together.

The implementation is written in Lean 4. Lean is designed as a general-purpose programming language with a native C backend and C FFI, which we use for the performance-critical build system implementations. Its macro system lets us embed qdt programs directly in Lean source files: since qdt’s syntax is a subset of Lean’s, inline test cases are Lean macros that produce qdt source strings. Lean’s dependent types make terms intrinsically scoped (the n in $\text{VTm } n$) and allow query result types to depend on the query key ($\text{Val} : \text{Key} \rightarrow \text{Type}$). The build system’s correctness proofs live in the same language as the implementation — the `Build` type carries its proof, and the elaborator’s tasks are the same object the proofs reason about.

3.1 Architecture

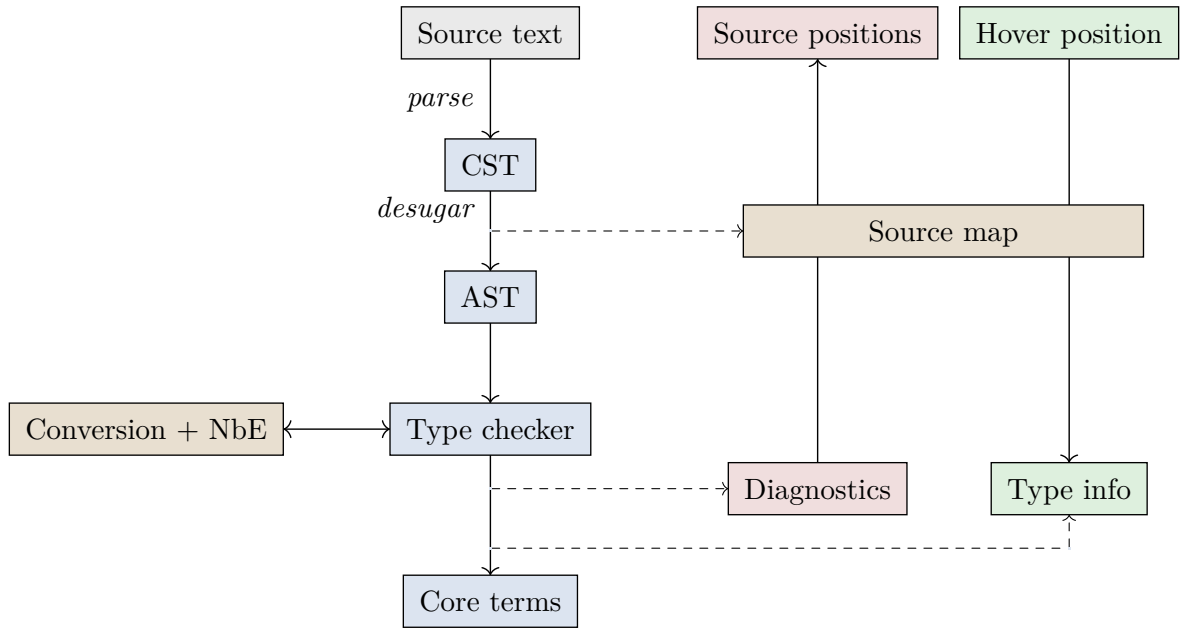


Figure 1: Elaboration pipeline. Source text is parsed to a CST, then desugared to an AST and a source map between CST and AST paths. The type checker produces core terms and emits diagnostics keyed by AST paths. Diagnostics are mapped through the source map to recover source positions.

The pipeline stages (Figure 1) are described in the following sections:

- **Parsing** (Section 3.2): a Pratt parser produces a green-tree CST, then desugars to an AST and a source map.
- **Bidirectional type checking** (Section 3.3): the AST is checked against expected types or synthesises its own, calling into NbE and conversion checking.
- **Normalisation by evaluation** (Section 3.4): terms are evaluated into a semantic domain using defunctionalised closures.
- **Conversion checking** (Section 3.5): decides definitional equality with rigid, flex, and full modes to avoid unnecessary unfolding.
- **Inductive types** (Section 3.6): declarations are elaborated into type formers, constructors, and recursors.

The remaining sections cover cross-cutting concerns:

- **Incremental elaboration** (Section 3.7): the build system decomposes elaboration into per-declaration queries, memoises results, and recomputes only what changed.
- **Language server** (Section 3.8): diagnostics and hover information are mapped through the source map and served to a VSCode extension.
- **Formal verification** (Section 3.10): three build systems are proven correct against batch evaluation.

Components are connected by query dependencies, not direct function calls. The parser produces a `cst` query result; the desugarer an `ast` query; each declaration’s elaborated form an `elabDecl` query depending on `constant` queries for every name it references. When a file changes, only the affected queries are recomputed.

To illustrate how the components interact, consider the elaboration of two definitions:

```
def id.{u} (A : Type u) (x : A) : A := x
def foo : Type 1 := id.{2} (Type 1) Type
```

Parsing. The source is parsed into a green-tree CST, then desugared to an AST with two definition nodes at indices 0 and 1. The `declarationIndex` query maps the name `id` to index 0 and `foo` to index 1.

Elaborating `id`. The `elabDecl` query for `id` fires. The universe parameter list is `[u]`. The parameter telescope is elaborated left to right:

- `A : Type u` — the annotation is a valid universe, so `A` is bound with type Type_u .
- `x : A` — the annotation refers to the parameter just bound. After evaluation, `x` is bound with type `A`.

The return type `A` refers to the first parameter. The body `x` is checked against `A` by context lookup. The elaborated constant has type $\Pi(A : \text{Type}_u). \Pi(x : A). A$ and body $\lambda A. \lambda x. x$.

Elaborating `foo`. The `elabDecl` query for `foo` fires. The expected type Type_1 is checked (it has type Type_2). The body `id.{2} (Type 1) Type` is an application, handled by `inferTm`:

1. **Infer the head.** `id` triggers `fetchConstant`, issuing a `constant` query and recording a dependency. The type is instantiated at level 2: $\Pi(A : \text{Type}_2). \Pi(x : A). A$. Evaluation produces a glued value: a neutral head `.const id [2]` paired with name and universe arguments for deferred unfolding.
2. **Check the first argument.** `Type 1` is checked against Type_2 . Inferred type: Type_2 . Conversion: $\text{Type}_2 = \text{Type}_2$ — success. The codomain closure is evaluated with `A` bound to Type_1 , yielding $\Pi(x : \text{Type}_1). \text{Type}_1$.
3. **Check the second argument.** `Type (Type0)` is checked against Type_1 . Inferred: Type_1 . Conversion: $\text{Type}_1 = \text{Type}_1$ — success.
4. **Subsumption.** Inferred result Type_1 against expected Type_1 . Conversion succeeds.

No delta-reduction of `id` was needed — the result type follows entirely from `id`'s Π type, without inspecting its body. The glued value was never forced.

Query dependencies. `elabDecl foo` depends on `constant id`. If `id`'s body later changes but its type remains $\Pi(A : \text{Type}_u). \Pi(x : A). A$, the hash is unchanged and `foo` is not recomputed. Only if `id`'s type changes does `foo` re-fire.

3.2 Parsing

The parser is a hand-rolled recursive descent Pratt parser [14], implemented as monadic combinators over `EStateM ParseError State`. Source text is first parsed to a concrete syntax tree (`Cst`) retaining trivia and exact token widths, then desugared to an abstract syntax tree (`Ast`) that strips trivia and expands surface syntax.

3.2.1 Green trees

The CST follows the *green tree* pattern [4], [15]. Nodes and tokens are tagged by `SyntaxNodeKind`; positions are not stored:

```

inductive Cst : Type
| token (kind : SyntaxNodeKind) (val : String)
| node (kind : SyntaxNodeKind) (children : Array Cst)
with
@[computed_field]
width : Cst → Nat
| .token _ val ⇒ val.length
| .node _ children ⇒ children.map width ▷ .sum

```

Nodes store only their kind and children, not absolute positions. Positions are recovered on demand by summing widths from the root. Because nodes lack positions, two identical subtrees are structurally equal regardless of where they appear in the file, making the CST Hashable. This is the key property for incrementality: a definition’s subtree hashes the same no matter where it sits in the file.

Separately, whitespace and comments are parsed as *trivia* tokens, stored as siblings of content tokens. Lowering discards trivia when producing the AST, so whitespace edits that only affect trivia tokens do not change the AST hash.

3.2.1.1 Example

Consider the file containing:

```

def a : Nat := Nat.zero
def b : Nat := a

```

The CST is a tree rooted at a `file` node, with trivia (whitespace, comments) and declaration subtrees as children. Each declaration subtree contains its tokens. Figure 2 shows the structure, with token children of each declaration abbreviated.

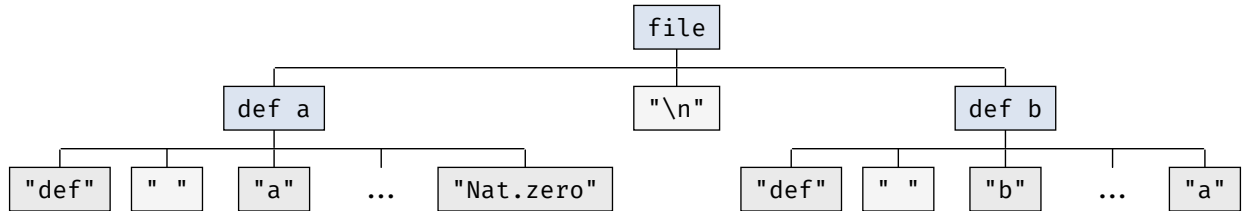


Figure 2: CST for the two-definition file. Whitespace is stored in separate trivia tokens.

3.2.1.2 Trivia separation and whitespace edits

If the user adds extra spaces within `def a`, only the affected trivia token changes. The content tokens (`"def"`, `"a"`, `" := "`, `"Nat.zero"`) are structurally identical. The CST hash does change (the trivia token is different), but lowering discards trivia, so the AST hashes the same as before. No downstream query is invalidated. This is a property of trivia separation in the CST and lowering, not of green trees per se.

3.2.1.3 Position independence and insertion

Green trees store no absolute positions. A definition’s subtree is determined entirely by its tokens and their structure, not by where it sits in the file. This means inserting a new definition `c` before `b` does not invalidate `b`’s elaboration: `b`’s subtree is structurally identical before and after the insertion.

The elaboration pipeline uses *paths* — lists of child indices relative to a subtree root — rather than absolute positions. A path `[2, 1]` means “child 2 of the subtree, then child 1 of that”.

Diagnostics and hovers are keyed by path relative to the declaration’s subtree. After the insertion, the `declarationIndex` query (which maps names to indices) recomputes — `b` is now at a different index — but `b`’s subtree hashes the same, so `elabDecl b` is not recomputed. Only `c` is elaborated.

If nodes stored absolute byte offsets, every node after the insertion point would have a different offset, producing a structurally different tree. Every query depending on those nodes would recompute, even though the source code is semantically unchanged.

3.2.2 Language server integration

Desugaring produces, alongside the AST, a `SourceMap` that bidirectionally maps paths in the CST to paths in the AST:

```
structure SourceMap where
  cstToAst : HashMap Path Path
  astToCst : HashMap Path Path
```

The map is populated during desugaring: as each AST node is constructed from a CST node, an entry is added to both tables. The language server uses it in both directions:

- *Hover*: given a cursor position, the CST is walked to find the path of the token under the cursor. `cstToAst` maps this to the AST path. Hover content is keyed by AST path, so the lookup is direct.
- *Diagnostics*: the elaborator records diagnostics at AST paths. `astToCst` maps these back to CST paths, which are converted to source spans by summing widths from the root.

Source positions are reconstructed only at this final step. The entire elaboration pipeline operates on paths, never on byte offsets. Path-based lookup is $O(\text{depth})$ rather than $O(\text{size})$.

3.2.3 Error recovery

On a parse error, a diagnostic is emitted and the parser skips to the next top-level declaration boundary (`def`, `inductive`, `axiom`, etc.). A single malformed declaration does not prevent the rest of the file from being elaborated.

A separate converter from `Lean.Syntax` to `Ast` via the metaprogramming framework is available for inline test cases in Lean source files, but is not used by the main pipeline.

3.3 Bidirectional type checking

Type checking is bidirectional [9]. Rather than a single judgement $\Gamma \vdash t : A$, the algorithm splits into two mutually recursive functions:

- `inferTm ctx ast : OptionT ElabM (Tm n × VTy n)`: synthesise a core term and its type. Used when the expected type is unknown.
- `checkTm ctx expectedTy ast : ElabM (Tm n)`: produce a core term at a known expected type. On failure, a diagnostic is emitted and a fresh axiom inserted as placeholder.

The split means a lambda’s parameter type comes from the expected `Pi` type (so it can be left unannotated), the algorithm is syntax-directed (one case per AST constructor), and mismatches are reported at the exact subterm.

The mode-switching rules:

- **Lambdas** are checked against $\Pi(x : A).B$: the body is checked against B with $x : A$ in context. If the parameter has a type annotation, it must match A ; otherwise A is taken from the expected type.
- **Applications** are inferred. The function f is inferred to type T . If T is not Pi after whnf , the elaborator fails. Otherwise $T = \Pi(x : A).B$, the argument a is checked against A , and the result has type $B[a/x]$.
- **Variables** are inferred by context or environment lookup.
- **Let-bindings**: infer the bound value's type, check the body in the extended context.
- **Universes** Type_ℓ are inferred to have type $\text{Type}_{\ell+1}$.
- **Subsumption**: when inference meets a checking position, the inferred type is compared against the expected type by the conversion checker.

`inferTm` and `checkTm` are mutually recursive with `checkIdent` for variable lookup (local context first, then global environment). The application case of `inferTm`:

```
| .node `Term.app #[fn, arg] => do
  let (fnTm, fnTy) <- inferTm ctx fn
  let .pi _ dom (env, codTm) := fnTy
  | throwError (.expectedFunctionType ctx.names (<- fnTy.quote))
  let argTm <- checkTm ctx dom arg
  let argVal <- argTm.eval ctx.env
  let codVal <- codTm.eval (env.cons argVal)
  return (.app fnTm argTm, codVal)
```

`inferTm` always returns a `VTy` produced by `Ty.eval`, so `fnTy` is already in weak head normal form; no `whnf` call is needed to expose the Pi . The codomain is substituted via evaluation in the closure's extended environment, not explicit syntactic substitution.

On failure (sorry, unbound variable, type mismatch) the elaborator emits a diagnostic and inserts a fresh axiom as placeholder. The rest of the file remains type-checkable, so the language server can report errors throughout.

3.4 Normalisation by evaluation

The conversion rule requires deciding definitional equality, which in turn requires reducing terms. The naive approach, repeatedly rewriting syntax until a normal form is reached, performs substitution one variable at a time, each time traversing the term. Normalisation by evaluation (NbE) [10] avoids this by interpreting syntax into a *semantic domain* of values, where substitution happens in bulk through environments rather than syntactic traversal.

Evaluation maps syntax into values; *quotation* (or *readback*) maps values back to syntax. The round trip produces a normal form.

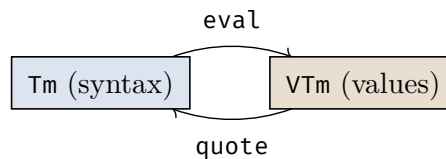


Figure 3: The NbE round trip.

In a type checker, we rarely need full normal forms. Most conversion checks succeed or fail after examining only the outermost constructor. So evaluation stops at *weak head normal form* (WHNF): it reduces the head of a term until it is a lambda, a Pi, a universe, or an irreducible variable, then stops. Subterms are reduced only on demand, when the conversion checker descends into them.

3.4.1 Values and variables

Values represent terms in WHNF. A value is either canonical (lambda, Pi, universe) or *neutral*: a variable or constant applied to a spine of arguments that cannot reduce further because the head is blocked.

Variables are represented using de Bruijn *levels* rather than *indices*. An index counts inward from the binding site: in $\lambda x. \lambda y. x$, the variable x has index 1. A level counts outward from the root of the context: x has level 0. The difference matters when opening a closure under a binder: adding a new variable to the context shifts every index but changes no level. Since evaluation constantly opens closures, levels avoid the cost of traversing values to adjust variable references.

3.4.2 Closures

A closure represents a function body waiting for its argument. In *higher-order abstract syntax* (HOAS), a closure is a host-language function $\text{VTm} \rightarrow \text{VTm}$: applying it is a native function call. In *defunctionalised* closures, a closure is a pair of the body's syntax and the captured environment: applying it re-evaluates the body in the extended environment.

HOAS is generally faster, but requires a non-strictly-positive inductive type ($\text{lam} : (\text{VTm} \rightarrow \text{VTm}) \rightarrow \text{VTm}$), which Lean 4's kernel over-approximately rejects, due to the possibility of unsoundness. I use defunctionalised closures to keep the evaluator within the language's logic.

3.4.3 Evaluation

Evaluation interprets syntax in an environment mapping bound variables to their values. Variables look up their value in the environment. Lambdas and Pis capture the current environment to form a closure. Application of a lambda to an argument evaluates the body in the closure's environment extended with the argument. This is where substitution happens, without traversing the body's syntax. Let-bindings are handled the same way: the bound value is evaluated and added to the environment, then the body is evaluated.

Constants receive special treatment. A defined constant (one with a body) evaluates to a *glued* value: a pair of the folded form (the constant's name and universe arguments) and the information needed to unfold it. The body is not fetched during evaluation; it is deferred until *whnf* forces it. This lazy unfolding is important for incrementality: if the conversion checker compares two glued values with the same head and succeeds without forcing either side, no dependency on the definition body is recorded.

3.4.4 Weak head normalisation

whnf takes a value and reduces it until the outermost form is canonical or irreducible. For most values this is immediate. The interesting cases are glued values and recursor applications. When *whnf* encounters a glued value, it fetches the definition body, evaluates it with the appropriate

universe substitution, re-applies the accumulated spine of arguments, and continues reducing (δ -reduction). This is the only point at which a definition body is materialised. When a recursor is fully applied and its major premise is a constructor, ι -reduction fires: the constructor’s fields are substituted into the matching branch.

3.4.5 Quotation

Quotation converts values back to syntax. Closures are opened by applying them to a fresh variable at the current de Bruijn level, and the result is recursively quoted. Levels are converted back to indices during this process. Glued values quote to their folded form (the constant name), preserving definition boundaries rather than inlining bodies.

3.5 Conversion checking

Conversion checking decides definitional equality. The naive algorithm (reduce both sides to normal form, compare structurally) forces all reducible subterms regardless of whether comparison needs them.

The algorithm here, from Kovacs’s `smalltt` [11], bounds speculative work with a three modes:

- **Rigid**: the default. When two values have the same defined head, spines are compared speculatively in flex mode. If flex succeeds, no unfolding was needed. If it fails, both sides are unfolded and compared in full mode.
- **Flex**: entered from rigid’s speculative check. No definitions are unfolded; no fallback. Any mismatch causes immediate failure, returning control to rigid.
- **Full**: entered after a flex failure. All definitions are unfolded on encounter. Once in full, all recursive calls remain in full.

Figure 4 shows the transitions between these modes. At most one backtrack per subterm: rigid attempts a cheap flex check, and on failure commits to the expensive full check. Flex never unfolds, so the cost of a failed speculation is bounded by the matching prefix of the two spines.

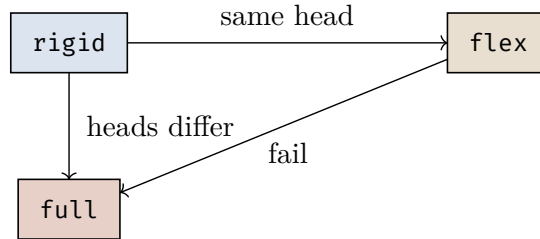


Figure 4: Conversion state transitions.

Consider comparing `f (g (h x))` with itself, where `f`, `g`, `h` are top-level definitions. The naive algorithm unfolds all three on both sides. The approximate algorithm observes the same folded structure: same head `f`, same spine. Flex descends into the argument, sees `g`, then `h`, bottoming out at `x` with no unfolding. The check runs in time proportional to the folded term, not the unfolded one.

To illustrate, consider comparing `Nat.add 2 3` against `Nat.add 2 3`. Both sides are glued values with the same defined head `Nat.add`. Rigid enters flex speculatively. Flex compares the spines elementwise: the first arguments are both `Nat.succ (Nat.succ Nat.zero)`, same constructor head `Nat.succ`; recurse into the field, same again, down to `Nat.zero` on both sides. The second arguments match similarly. Flex succeeds. No definition body was fetched.

Now comparing `Nat.add 2 3` against `5`. The heads differ: `Nat.add` (a defined constant, appearing as a glued value) versus `Nat.succ` (a constructor, appearing as a neutral). Different heads skip flex and enter full mode. Full forces the glued value via `whnf`, which fetches `Nat.add`'s definition body and fires iota-reduction on the recursor three times, yielding `Nat.succ (Nat.succ (Nat.succ (Nat.succ Nat.zero)))`. This matches `5`. The check succeeds, but `Nat.add`'s body was fetched, so a dependency is recorded.

3.5.1 Eta-conversion

Two cases are handled. *Function eta*: when comparing a lambda against a non-lambda (or two lambdas), both sides are applied to a fresh variable at the current de Bruijn level, and the bodies are compared. This decides $f \equiv \lambda x.f x$ without the term `f` needing to syntactically be a lambda.

Structure eta: when one side is a constructor application `c(r_1, ..., r_k)` of a single-constructor inductive (a structure) and the other is an arbitrary term `s`, each projection `s.i` is compared against the corresponding field `r_i`. This decides `c(s.1, s.2, ..., s.k) equiv s`: the constructor applied to all projections of `s` is definitionally equal to `s`.

An alternative, used in Lean's `isDefEq` and Agda's conversion checker, is *on-the-fly* reduction: `whnf` each side just enough to decide equality, recurse under binders. This avoids full normal forms, but without the rigid/flex distinction it unfolds whenever heads differ syntactically. The speculative flex check avoids this in the common case of matching defined heads.

3.6 Inductive types

An inductive declaration (Dybjer [16]) introduces a type former, its constructors, and a recursor. The elaboration proceeds in phases: elaborate the type and parameters, register the inductive as opaque (so constructors can refer to it), elaborate each constructor, check strict positivity and universe consistency, then generate the recursor type and its iota-reduction rules.

For `Nat`, the generated recursor is:

`Nat.rec.{u} : (C : Nat → Typeu) → C zero → ((n : Nat) → C n → C (succ n)) → (n : Nat) → C n`

with reduction rules:

$$\begin{aligned} \text{Nat.rec } C \ z \ s \ \text{zero} &\rightsquigarrow z \\ \text{Nat.rec } C \ z \ s \ (\text{succ } n) &\rightsquigarrow s \ n \ (\text{Nat.rec } C \ z \ s \ n) \end{aligned}$$

There is no primitive pattern matching; all computation on inductives proceeds through the recursor. Structures (single-constructor inductives) additionally generate projection functions and an eta rule: `c (s.f_1) ... (s.f_k)` is identified with `s`.

The motive's universe `u` is a fresh universe parameter, distinct from the inductive's own universe parameters. This allows elimination into any universe: `Nat.rec.{0}` gives recursion into `Type 0`, while `Nat.rec.{1}` gives large elimination into `Type 1`.

3.7 Incremental elaboration

Elaboration is decomposed into queries managed by a Shake-based build system. Each query has a tag and parameters; its result type is determined by the tag. `Key` has 17 cases; the principal ones:

```
inductive Key where
| cst : FilePath → Key
| ast : FilePath → Key
| declarationIndex : FilePath → Key
| elabCmdAt : FilePath → Nat → Key
| elabDecl : FilePath → Name → Key
| constant : FilePath → Name → Key
| lookupInfo : FilePath → Name → Key
-- ... others omitted

def Val : Key → Type
| .cst _           ⇒ Cst × Array ParseError
| .ast _           ⇒ Ast
| .declarationIndex _ ⇒ HashMap Name Nat × Array Diagnostic
| .elabCmdAt _ _    ⇒ Global × ElabInfo
| .elabDecl _ _     ⇒ Option (Constant × Origin) × ElabInfo
| .constant _ _     ⇒ Option (Constant × Origin)
| .lookupInfo _ _   ⇒ ElabInfo
-- ... others omitted
```

The principal queries:

- `declarationIndex`: maps declaration names to indices within the file. Depends only on the parsed AST.
- `elabDecl`: elaborates one declaration. Fetches the AST, looks up the declaration, type-checks it. Depends on `constant` queries for every referenced name.
- `constant`: the elaborated form of a named constant. This is the primary cross-declaration dependency.

When the elaborator encounters a constant reference, `fetchConstant` issues a `Key.constant` query, registering a dependency. An `entryCache` in `ElabState` memoises lookups within a single elaboration run to avoid repeated queries for the same name.

3.7.1 Query wiring

The `tasks` function maps each `Key` to its computation. Three representative cases illustrate how queries chain:

`constant filepath name` resolves a name to its elaborated form across files. It first tries the local file via `lookup filepath name`, which delegates to `elabDecl filepath name`. If that returns `none` (the name is not defined in this file), it fetches `transitiveImports filepath` to get the set of imported files, then tries `lookup path name` for each imported file until it finds a match. This two-phase lookup is the primary cross-file resolution mechanism: editing a definition in one file invalidates `constant` queries in other files that resolved to it.

`elabDecl filepath name` elaborates a single declaration. It fetches `declarationIndex filepath` (a map from names to command indices), looks up the index for `name`, then fetches `elabCmdAt filepath idx` to run the elaborator on that command. The result is the elaborated `Constant`

and diagnostic information. Because the index is looked up dynamically, reordering definitions in a file changes the index mapping and triggers re-elaboration of the affected declarations.

`checkFile filepath` aggregates all diagnostics for a file. It fetches `astSourceMap filepath` for parse errors, `declarationIndex filepath` for the list of declarations, then `lookupInfo filepath name` for each declared name to collect elaboration diagnostics. This is the top-level query that the CLI and language server invoke.

3.7.2 The elaboration monad

`ElabM` combines incremental queries with elaboration state:

```
abbrev ElabM := ReaderT ElabContext (StateT ElabState (Task Monad config ι₀  
q₀))
```

`Task` is the query monad: `Task.fetch` calls register dependencies. `StateT ElabState` threads the local environment, constant cache, and accumulated diagnostics and hovers. `ReaderT ElabContext` carries file path, declaration name, and universe parameters. `ι₀` and `q₀` identify the current query in the dependency graph.

3.7.3 Granularity of queries

If queries are too coarse, such as per-file granularity, any edit invalidates the whole file. If they are too fine, per-expression, the per-query overhead dominates.

I chose per-declaration granularity. `Key.elabDecl` elaborates one `def`, `inductive`, or `structure`. Dependent declarations are recomputed only if the edit changes the cached `Constant`.

Finer granularity would require making subterms addressable, adding engineering complexity for diminishing returns.

3.7.4 Dependently-typed query results

The dependent `Val : Key → Type` lets `fetch (Key.constant p n) : Task (Option Constant)` type-check without tagging; the compiler specialises each call site. The alternatives are a sum type (boilerplate matching at every consumer), an existential (`unsafeCast` at every fetch), or GADTs (where an index on the key type determines the result type, as in *Rock* and *Salsa*). Dependent types express this directly.

3.7.5 Query dependency graph

Figure 5 shows the query dependency graph for a two-definition file:

```
def foo : Type 1 := Type  
def bar : Type 1 := foo
```

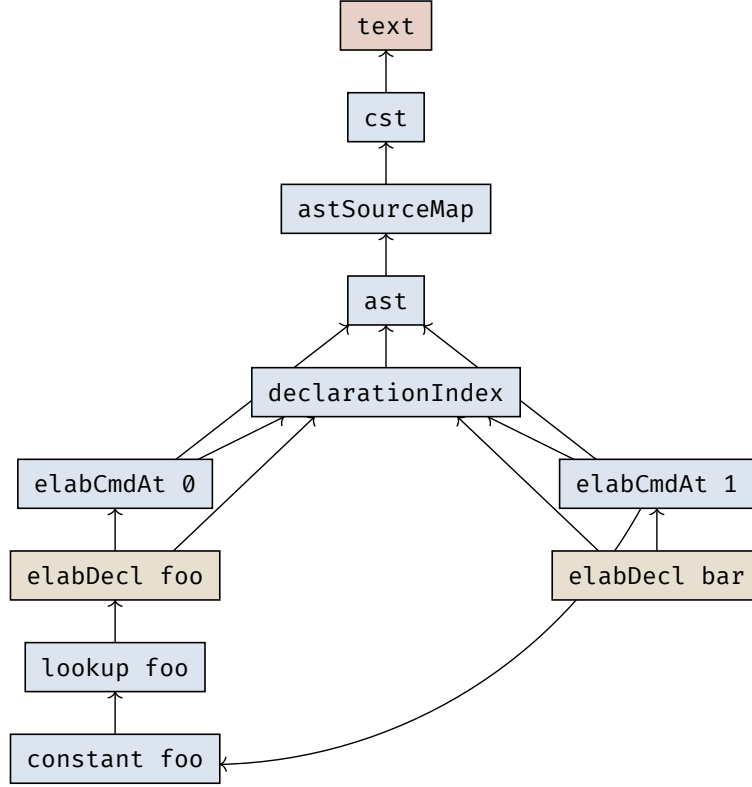


Figure 5: Query dependency graph for a file with two definitions. Arrows point from a query to its dependencies. The `text` node (red) is the input. Elaborating `bar` depends on `constant foo`, which depends on `elabDecl foo`. Editing `foo`’s body invalidates `elabCmdAt 0`, `elabDecl foo`, `lookup foo`, and `constant foo`; if the resulting `Constant` hashes the same (e.g. the type is unchanged), `elabDecl bar` is not recomputed.

3.7.6 Glued evaluation and dependency tracking

Glued evaluation (Section 3.4) produces values carrying a constant’s name and universe arguments without fetching its body. The body is fetched only when `whnf` forces the value. Flex mode compares two glued values with the same head without forcing either side: no body fetched, no dependency recorded. Only when heads disagree and full mode fires does delta-reduction fetch the body and record a dependency.

Editing a definition’s body thus invalidates only the queries that actually unfolded it. Call sites whose checks succeeded in flex mode have no dependency on the body.

3.7.7 The Shake algorithm

When the build system receives a request for query q , it executes the following procedure:

1. **Cache lookup.** Check if q has a cached `Memo` from a previous build.
2. **Verify input fingerprints.** For each input dependency recorded in the memo, hash the current input value and compare against the stored fingerprint. If any mismatch, go to step 5.
3. **Verify query fingerprints.** For each query dependency recorded in the memo, *recursively fetch* that dependency (which may itself verify or recompute), hash the result, and compare. If any mismatch, go to step 5.
4. **Cache hit.** All fingerprints match. Return the cached value.

5. **Recompute.** Execute the task, recording which inputs and queries it fetches. Hash the results. Store a new **Memo** with the value and its dependency fingerprints.

Figure 6 illustrates verification on a fragment of the query graph. Each edge is annotated with the fingerprint stored at build time. When `elabDecl bar` is fetched, Shake verifies its dependency `constant foo` by recursively fetching it, which in turn verifies `elabDecl foo` against the input `text`. If the input has changed, `elabDecl foo` is recomputed. If the new result hashes the same as the stored fingerprint on the edge to `constant foo`, verification of `elabDecl bar` still passes (early cutoff). If it hashes differently, `elabDecl bar` is recomputed too.

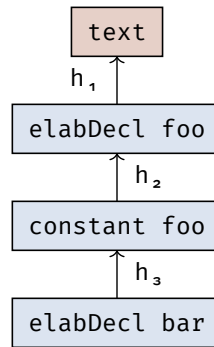


Figure 6: Shake verification. Each edge stores a fingerprint (h_1 , h_2 , h_3) from the previous build. To verify `elabDecl bar`, Shake recursively fetches `constant foo`, which fetches `elabDecl foo`, which checks h_1 against the current input. If h_1 mismatches, `elabDecl foo` is recomputed; if its new result still matches h_2 , `elabDecl bar` is not recomputed (early cutoff).

Verification is *suspending*: the build walks the graph demand-driven, recursing into each dependency before deciding whether the current query needs recomputation. This is the suspending scheduler with verifying traces in the terminology of Mokhov, Mitchell, and Peyton Jones [7].

3.7.8 Early cutoff

Shake hashes each query’s result alongside its dependency fingerprints. When a dependency is recomputed and its result hashes the same as before, the queries that depend on it are not recomputed; their fingerprints still match.

Consider appending a new definition to `Nat.qdt`. The `declarationIndex` query is invalidated (a new name appears), and the new definition’s `elabDecl` query runs. But the existing definitions — `Nat.add`, `Nat.succ`, etc. — produce the same `Constant` values as before. Their hashes are unchanged, so `constant Nat.add` returns the cached result. Files that import `Nat` see the same fingerprint and are not recomputed, even though `Nat.qdt` was edited.

Without early cutoff, any edit to `Nat.qdt` would cascade through every importing file. With early cutoff, the cascade stops at the first query whose result is unchanged.

3.7.9 Native build system implementations

The Lean Shake implementation (`Incremental/Shake.lean`) uses `runST` with mutable references for the memo table and in-progress cache. Two additional implementations in C — `shake.c` (622 lines) and `salsa.c` (519 lines) — implement the same `Build` interface via Lean’s `@[extern]` FFI.

The C implementations must match Lean’s runtime ABI: constructor field order and scalar layout for `Memo` and `Store`, closure arity for the `fetch` and `input` callbacks passed to tasks, and reference counting protocol for all allocated objects.

The C implementation avoids Lean’s closure allocation and monadic bind dispatch on each query.

3.8 Language server

The elaborator doubles as a language server. Hover information and diagnostics are collected in `ElabState` during elaboration and served to a VSCode extension via LSP:

- **Diagnostics:** type errors, unbound variables, universe mismatches — positioned via path indices into the AST.
- **Hover:** the type under the cursor, or the full signature of a referenced constant.

3.8.1 Incremental test harness

The test suite verifies incremental correctness via a harness simulating editor interactions:

- `setText text filepath` sets the contents of a file and triggers a full rebuild through the query system.
- `diagnostics check filepath` asserts that the diagnostics for a file satisfy a predicate.
- `hover pos expected start stop` asserts that hovering at a given position produces the expected content and source span.

Tests sequence multiple `setText` calls and check that diagnostics and hovers update correctly. The cases cover pathological incremental scenarios: swapping definitions so forward references become backward; renaming and checking that dependents report unbound variables; atomically moving a definition between files; and cyclic imports. These exercise dependency tracking, cache invalidation, and error recovery paths that batch elaboration would not stress.

3.9 Repository overview

Directory	Purpose	Lines	Section
Incremental/	Build system framework: Task, Build, BuildConfig, Busy, LessBusy, Shake, Salsa, free theorem, plus ShakeCPS variants	1,209	2.3
Incremental/c/	C implementations of Shake and Salsa (2 files)	1,141	3.7
Qdt/Frontend/	Parser, CST, AST, desugarer (4 files)	1,371	3.2
Qdt/Theory/	Declarative type theory: syntax, contexts, typing rules, universes, substitution, weakening, alpha-equality (11 files)	1,907	2.1
Qdt/	Bidirectional checker, NbE, conversion, inductives, quoting, control monads, query types, task rules, error handling, pretty printing	2,804	3.3–3.7
Qdt/Test/	Elaborator regression tests (12 files)	610	4.1
Qdt/Lsp/Test/	Incremental tests	438	3.8
FSWatch/	File system watcher	361	—
Main.lean	CLI entry point	173	—
Lsp.lean	Language server entry point	330	3.8

Table 1: Source code by directory. Line counts computed with `wc -l`, excluding `.lake` and `stdlib` examples. Total: 9,203 lines of Lean + 1,141 lines of C.

3.10 Formal verification

The build system framework is formalised in Lean 4 with machine-checked proofs. We refine the “Build Systems à la Carte” (BSALC) framework [7] in three ways, each forced by the requirements of a dependently typed elaborator.

3.10.1 Refinements over BSALC

BSALC uses a single key type k with a uniform value type v . A type checker needs different result types for different queries: parsing returns a syntax tree, elaboration returns a typed term, diagnostics are an array of errors. Without dependent result types, every fetch would require a runtime cast. We separate the key space into *inputs* I (file contents, set externally) and *queries* Q (computed results), each with a dependent value type:

$$V : I \rightarrow \text{Type}, \quad R : Q \rightarrow \text{Type}$$

BSALC does not address termination: the Haskell implementation simply assumes tasks do not cycle. For a total specification we need a well-foundedness condition. The dependency relation is indexed by the current input state, because different source files can induce different dependency orders among queries:

TODO: No it isn’t

$$\text{rel} : (\prod_i V(i)) \rightarrow Q \rightarrow Q \rightarrow \text{Prop}, \quad \text{wf} : \forall \iota. \text{WellFounded}(\text{rel}_\iota)$$

These are bundled into a configuration:

```

structure BuildConfig : Type 1 where
  I : Type; V : I → Type
  Q : Type; R : Q → Type
  rel : (∀ i, V i) → Q → Q → Prop
  wf : ∀ i, WellFounded (rel i)

```

TODO: fix

A task for query q_0 under input state ι_0 is polymorphic in an effect f . It receives two callbacks: one to read inputs, one to fetch query results. The fetch callback requires a proof that the fetched query precedes q_0 in the dependency relation:

```

def Task (α : Type) : Type 1 :=
  ∀ (f : Type → Type) [c f],
    (∀ i, f (℄.V i)) →
    (∀ q, ℄.rel  $\iota_0$  q q $_0$  → f (℄.R q)) →
    f α

```

The polymorphism in f is the key design: the elaborator’s tasks are written once, and the build system chooses the monad. `compute` (the batch specification) instantiates f with `Id`; `LessBusy` and `Shake` instantiate it with `StateM Cache`. The free theorem (Section 3.10.3) exploits this polymorphism to prove they produce the same result.

3.10.2 Correctness

A correct build system produces the same result as `compute`, a reference semantics defined by well-founded recursion. `compute` runs each task in `Id` (the trivial monad, no effects), recursively recomputing every dependency from scratch:

```

def compute (tasks : Tasks c ℄) ( $\iota$  : ∀ i, ℄.V i) (q : ℄.Q) : ℄.R q :=
  tasks q Id  $\iota$  (fun q' _ => compute tasks  $\iota$  q')
termination_by ℄.wf.wrap q

```

A build system carries private state σ and returns a value with a proof that it equals `compute`:

```

structure Build (c) (℄ : BuildConfig) (J : Type) [Input ℄ J] where
  σ : Type
  init : J → σ
  build :
    (tasks : Tasks c ℄) → (q : ℄.Q) → (store : σ) →
    { r : ℄.R q // r = compute tasks (inputs store) q } × σ

```

Any inhabitant is correct by construction. The elaborator’s `tasks` value is defined once and works with any `Build` inhabitant. This separation means the build strategy can be swapped without touching the elaborator: one can use a verified Lean implementation during development and a fast C implementation in production, confident that both compute the same results. Three build systems inhabit this type:

Busy calls `compute` directly. Correctness follows by definition.

```
def Busy : Build c @ J where
  build tasks q store :=
    (<compute tasks (Input.get store) q, rfl>, store)
```

3.10.3 The free theorem

LessBusy and Shake run tasks in `StateM Cache`, a monad threading a cache of previously computed results. However correctness is stated against `compute`, which runs tasks in `Id` (the trivial monad with no effects). The free theorem establishes that a task produces the same result in both monads, provided the fetches return the same values.

We argue for the correctness of these build systems via *parametricity* (Wadler [17]). For example, a function of type $\forall a, \text{List } a \rightarrow \text{List } a$ can only rearrange, duplicate, or drop elements — it cannot inspect them, because a is abstract. This forces it to commute with `map`: whatever positions it selects, it selects the same way regardless of what occupies them. A `Task` is polymorphic in its monad in the same way: it can only sequence fetches and combine results, never inspect which monad it runs in. If the fetches return the same values in both monads, the task produces the same final result.

To formalise this, we define a `MonadAction` relating two monads — a family of relations on their carrier types, preserving `pure` and `>=>`:

```
structure MonadAction (κ₁ κ₂ : Type → Type) [Monad κ₁] [Monad κ₂] where
  rel : (α → β → Prop) → κ₁ α → κ₂ β → Prop
  rel_pure : R a b → rel R (pure a) (pure b)
  rel_bind : rel R ma mb → (∀ a b, R a b → rel S (ka a) (kb b))
    → rel S (ma >=> ka) (mb >=> kb)
```

The free theorem then states: if inputs and fetches are pointwise related, the task result is related:

```
axiom Task.freeTheorem :
  (hι : ∀ i, F.rel Eq (ι₁ i) (ι₂ i)) →
  (hfetch : ∀ q hq, F.rel Eq (fetch₁ q hq) (fetch₂ q hq)) →
  F.rel Eq (t κ₁ ι₁ fetch₁) (t κ₂ ι₂ fetch₂)
```

Since `Task` is polymorphic in a type constructor f constrained by `Monad`, this requires higher-kinded parametricity (Voigtländer [18]). Parametricity cannot be proved internally in Lean; it is stated as an axiom. The axiom is incompatible with `Classical.choice`; the formalisation avoids choice throughout.

From the axiom, we derive `Tasks.freeTheorem`: specialising to $\kappa_1 := \text{StateM Cache}$ and $\kappa_2 := \text{Id}$, if the cache provides the same values as `compute` for every fetch, the overall task result equals `compute`. The proof unfolds `compute` once and applies the axiom.

3.10.4 LessBusy

Busy recomputes every dependency from scratch, exponential in the depth of the dependency graph. LessBusy avoids this by caching results within a single build, so each query is computed at most once. Each cache entry is a dependent pair $\{ r : \mathbb{C}.R \ q \ // \ r = \text{compute tasks } \iota$

`q }`, bundling a value with its correctness proof. The `fetch` function checks the cache first; on a miss, it runs the task, caches the result, and returns it:

```
def fetch (qo : ℄.Q) :
  StateM VCache (Value qo) := do
  if let some v := (← get).get? qo then return v
  let v ← run qo (fun q hq ⇒ fetch q)
  modify (·.insert qo v)
  return v
termination_by ℄.wf.wrap qo
```

Termination follows from the well-founded relation: each recursive `fetch` call provides a proof $\mathbb{C}.\text{rel } q' \ q_0$, so the recursion decreases. The `run` function executes the task and produces a proven `Value`; this is where the free theorem is used.

The proof constructs a concrete `MonadAction (StateM VCache) Id`:

```
def action : MonadAction (StateM VCache) Id where
  rel P m b := ∀ s, P (m s).1 b
```

This says: a stateful computation `m` is related to a pure value `b` when, from any starting state, `m`'s return value satisfies the relation with `b`. Each cached `fetch` returns the same value as `compute` (the cache entries carry their own proofs), so `Tasks.freeTheorem` gives the overall result.

The top-level definition ties `fetch` into the `Build` type:

```
def LessBusy : Build Monad ℄ J where
  build tasks q store :=
    let ιo := Input.get store
    let (v, _) := fetch tasks ιo q (DHashMap.empty)
    (v, store)
```

The empty cache is passed as the initial state. Each `fetch` call populates it, and the returned `v` carries a proof that it equals `compute`.

3.10.5 Shake

`LessBusy` starts fresh on each build; the cache is discarded between invocations. `Shake` fills this gap by persisting the cache across builds, using fingerprints to determine which entries are still valid. Each cached `Memo` carries a universally quantified invariant:

```
structure Memo (q : ℄.Q) where
  value : ℄.R q
  inputDeps : List ((i : ℄.I) × H)
  deps : List (Σ' (q' : ℄.Q) ( _ : ℄.rel q' q), H)
  invariant :
    ∀ (ι : ∀ i, ℄.V i),
      (∀ p ∈ inputDeps, hI p.1 (ι p.1) = p.2) →
      (∀ p ∈ deps, hR p.1 (compute tasks ι p.1) = p.2.2) →
      value = compute tasks ι q
```

The invariant says: for *any* input function ι , if every recorded input fingerprint matches ι and every recorded dependency fingerprint matches `compute` under ι , then `value` equals `compute`

$\text{tasks } \iota \ q$. This is universally quantified over ι : the same memo is valid across builds with different inputs, as long as the fingerprints match.

On a cache hit, `verifyInputs` checks each recorded input fingerprint against the current inputs, and `verifyDeps` recursively fetches each dependency (getting a proven `Value`) and checks its fingerprint. If both pass, the memo’s `invariant` directly gives the correctness proof.

On a cache miss, `runRecompute` evaluates the task via a free monad tree and builds a fresh `Memo`. The invariant proof uses `FM.evalTree_cross`: if two sets of inputs and dependencies agree at every position recorded in the trace (checked via injective embeddings $hI : \mathbb{C}.V \ i \hookrightarrow H$ and $hR : \mathbb{C}.R \ q \hookrightarrow H$), the free monad tree evaluates to the same result. The injectivity of the hash functions (`Function.Embedding`) is what makes fingerprint comparison sound: $\text{hash } a = \text{hash } b$ implies $a = b$. In practice, the `Hashable` instances use 64-bit hashes that are not injective; collisions are possible but astronomically unlikely. The proof assumes an ideal hash function, an injective function into 64 bit integers.

The elaborator’s tasks are a single `Tasks` value, independent of which `onBuild` executes them — incremental elaboration is provably equivalent to `compute`.

The formalisation comprises approximately 1,500 lines of Lean across the core library (`Basic`, `Busy`, `LessBusy`, `Shake`, `FreeTheorem`, `FreeMonad`).

Chapter 4

Evaluation

We evaluate correctness, performance, and incrementality.

4.1 Correctness

The primary test is the standard library: 2,300 lines of qdt code across 39 files, covering natural number arithmetic, propositional equality, well-founded recursion, sigma types, monadic abstractions, and the Ackermann function, via well-founded recursion.

Equality proofs exercise the conversion checker: `Eq.refl.{0} Nat 6` at type `Nat.add 2 4 = 6` succeeds only if the elaborator correctly reduces `Nat.add 2 4` to 6 via iota-reduction.

4.2 Batch elaboration performance

TODO: re-measure on final code with hardware spec.

4.3 Conversion checking performance

We stress-test the evaluator and conversion checker with Church-encoded benchmarks from Kovacs’s normalisation-bench [11]. These encode natural numbers as lambda terms, then check definitional equality between two independently computed values of the same normal form, forcing n beta-reductions on each side.

Benchmark	Time (ms)
Nat 10K	176
Nat 100K	1,708
Nat 1M	16,417

Table 2: Church numeral conversion checking. Time scales linearly with n .

For comparison, Kovacs’s normalisation-bench [11] reports `Nat 5M conv` in 90ms for a GHC-compiled HOAS evaluator, and 500ms for smalltt (which does more bookkeeping for elaboration). The gap with our implementation is due to defunctionalised closures: each beta-

reduction re-interprets the body in an extended environment, whereas HOAS compiles the closure to a native function call.

4.4 Scaling benchmarks

We compare wall-clock time between qdt and Lean 4 on synthetic programs of increasing size. Lean’s time includes kernel checking, which qdt does not perform. Each generator produces a file with N definitions following a specific dependency pattern:

- *Discrete* ($n_i := \text{Nat.zero}$): independent definitions, isolates per-definition overhead.
- *Random* ($n_i := \text{Nat.succ } n_j$ for random $j < i$): realistic dependency shape.
- *Chain* ($n_i := \text{Nat.succ } n_{i-1}$): linear dependency chain, stresses recursive query resolution.
- *Triangle* ($n_i := n_0 + n_1 + \dots + n_{i-1}$): conversion-heavy, each definition reduces a sum of predecessors.

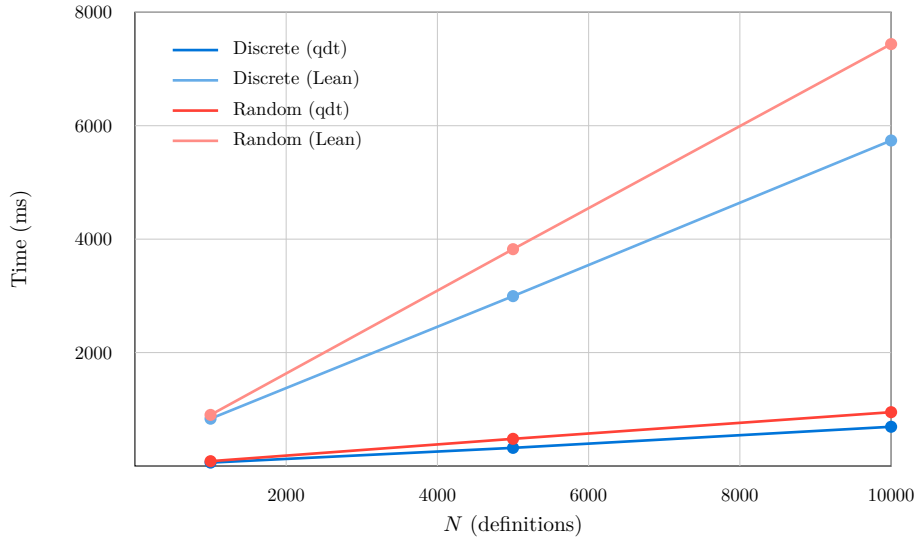


Figure 7: Discrete and Random scaling: qdt vs Lean 4. Lean’s time includes startup and kernel checking.

Benchmark	N	qdt (ms)	Lean (ms)
Discrete	1,000	60	834
Discrete	5,000	320	2,995
Discrete	10,000	692	5,739
Random	1,000	85	901
Random	5,000	479	3,823
Random	10,000	949	7,439
Chain	100	17	368
Chain	500	127	672
Chain	1,000	342	952
Triangle	25	22	463
Triangle	50	101	964
Triangle	75	274	1,872
Triangle	100	587	3,074
Triangle	150	1,835	6,477

Table 3: Full scaling results. Chain and Triangle use smaller N due to superlinear growth.

4.5 Incremental re-elaboration

We measure re-elaboration time after targeted edits on the standard library, using a retained Shake store. The Shake build system is instrumented to count cache hits (queries whose fingerprints match and are reused) and recomputed queries. After a cold build populates the store, each edit modifies one input and triggers a rebuild against the existing cache.

Edit	Time (ms)	Speedup
Cold build (no cache)	1,027	$1 \times$
No-op (retained store, no changes)	129	$8.0 \times$
Whitespace (append newline to entry file)	127	$8.1 \times$
Leaf (append definition to Ackermann.qdt)	172	$6.0 \times$
Hub (append definition to Nat.qdt)	170	$6.0 \times$

Table 4: Incremental re-elaboration of the standard library after targeted edits. Speedup is relative to cold build.

The no-op rebuild verifies every fingerprint in the store but recomputes nothing — the 129ms is the cost of traversing the dependency graph and checking hashes. The whitespace edit has the same cost: the green tree representation absorbs the whitespace change, the AST hashes the same, and no downstream query is invalidated.

The leaf edit appends a new definition to `Ackermann.qdt`, a file that no other file imports. The `declarationIndex` query for that file is invalidated (a new name appears), and the new definition is elaborated, but no other file’s queries are affected. The hub edit appends a definition to `Nat.qdt`, which is imported by five other files. Despite the wider dependency fan-out, the rebuild takes the same time: the new definition does not change any existing constant’s type

or body, so the `constant` queries for existing `Nat` definitions hash the same, and early cutoff prevents the cascade from propagating to dependent files.

4.6 Discussion

The incremental results demonstrate that the query-based architecture achieves its goal: after an edit, re-elaboration time is proportional to the amount of work that actually changed, not the size of the file or the number of dependents. The $6\text{--}8\times$ speedup over cold build on the standard library reflects the fact that most queries are unchanged after a typical edit.

The scaling benchmarks show that `qdt`'s per-definition elaboration cost is competitive with Lean 4 (which additionally performs kernel checking). The conversion checker scales linearly on Church-encoded benchmarks, with a constant-factor gap from the defunctionalised closure representation.

The formal verification ensures that incremental results are provably equivalent to batch evaluation. The free theorem bridges the gap between the memoising build systems and the specification, with all proof obligations discharged except the parametricity axiom.

Chapter 5

Conclusion

5.1 Summary

This project set out to build an incremental elaborator for a dependently typed language, using a formalised build system to guarantee that incremental results agree with batch elaboration. The elaborator supports dependent function types, Tarski-style universes with universe polymorphism, inductive types with recursors, and structures with projections, and successfully elaborates a 2,300-line standard library.

5.2 Returning to the claims

The introduction made four claims. I revisit each:

1. *Existing systems re-check entire file suffixes after an edit* (Section 4.5). The incremental elaborator avoids this: changing `Nat.add`'s body in a file imported by five others triggers re-elaboration of only the affected queries. Early cutoff prevents the cascade from propagating when a recomputed result hashes the same.
2. *No mainstream proof assistant uses query-based incrementality*. This remains true. The project demonstrates that query-based incrementality is viable for a dependently typed language, with per-declaration granularity and dynamic dependency tracking.
3. *Fredriksson's Sixty does not formalise the underlying build system*. The formalised build system framework (Section 3.10) provides what Sixty lacks: three build systems (Busy, LessBusy, Shake) proven correct by construction, with the free theorem ensuring that a task polymorphic in its monad produces the same result under any build strategy.
4. *The dependency structure is discovered dynamically during elaboration*. Glued evaluation and approximate conversion checking (Section 3.5) reduce the dependencies recorded: flex-mode comparison avoids fetching definition bodies, and early cutoff prevents propagation when results are unchanged.

5.3 Extensions beyond the proposal

The original proposal specified using the Salsa framework as a black box. Instead, the build system was formalised from scratch with machine-checked correctness proofs. Glued evaluation

and approximate conversion checking (rigid/flex/full) were added to reduce the dependency graph. A language server with diagnostics and hover information was implemented. Structures with projections and eta-expansion were added to support the standard library’s algebraic hierarchy.

5.4 Lessons learnt

Language pivots. The project moved from Rust to OCaml to Lean 4. Each transition was costly, but Lean’s dependent types proved essential: intrinsically scoped terms ($\text{V}\text{Tm } n$), dependently typed query results ($\text{Val} : \text{Key} \rightarrow \text{Type}$), and proof-carrying build systems ($\{ r // r = \text{compute } \dots \}$) would have required unsafe casts or boilerplate in the other languages.

Verification methodology. The free theorem is stated as an axiom in Lean because parametricity cannot be proved internally. This is a limitation of the formalisation, but the axiom is well-established in the literature (Reynolds, Wadler, Voigtlander, Atkey) and its use is confined to a single lemma.

Performance. The elaborator is approximately 100x slower than `smalltt` on normalisation-heavy benchmarks, due to Lean’s reference-counting runtime overhead on closure-heavy code. This is a constant factor; the asymptotic behaviour is the same. For interactive use, the incremental rebuild avoids most of this cost.

5.5 Future work

- **Parallelism.** Mokhov et al. [19] introduce *selective functors*, sitting between `Applicative` and `Monad`. Selective tasks could allow the build system to parallelise queries with static dependencies (e.g. parsing independent files) while retaining dynamic dependencies for elaboration.
- **Cancellation.** A language server should cancel in-progress elaboration when the user edits again. A middleware framework could support this via an exception monad that preserves partial progress in the memo store.
- **Pattern matching.** The core theory uses recursors directly. A pattern-matching compiler translating case trees to recursor applications would make the surface language more practical without changing the core.

Bibliography

- [1] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, Springer, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.
- [2] U. Norell, “Dependently Typed Programming in Agda,” in *Advanced Functional Programming*, Springer, 2009, pp. 230–266. doi: 10.1007/978-3-642-04652-0_5.
- [3] B. Barras *et al.*, “The Coq Proof Assistant Reference Manual.” 1999.
- [4] A. Kladov and contributors, “rust-analyzer.” [Online]. Available: <https://rust-analyzer.github.io/>
- [5] N. Matsakis and contributors, “Salsa: A generic framework for on-demand, incrementalized computation.” [Online]. Available: <https://github.com/salsa-rs/salsa>
- [6] O. Fredriksson, “sixty.” [Online]. Available: <https://github.com/ollef/sixty>
- [7] A. Mokhov, N. Mitchell, and S. P. Jones, “Build Systems à la Carte: Theory and Practice,” *Journal of Functional Programming*, vol. 30, p. e11, 2020.
- [8] P. Martin-Löf, *Intuitionistic Type Theory*. Naples: Bibliopolis, 1984.
- [9] J. Dunfield and N. Krishnaswami, “Bidirectional Typing,” *ACM Computing Surveys*, vol. 54, no. 5, 2021, doi: 10.1145/3450952.
- [10] A. Abel, “Normalization by Evaluation: Dependent Types and Impredicativity,” *Lecture notes*. 2013.
- [11] A. Kovács, “smalltt.” [Online]. Available: <https://github.com/AndrasKovacs/smalltt>
- [12] U. A. Acar, G. E. Blelloch, and R. Harper, “Adaptive Functional Programming,” in *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 2002, pp. 247–259.
- [13] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, “Adapton: Composable, Demand-Driven Incremental Computation,” in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2014, pp. 156–166.
- [14] V. R. Pratt, “Top Down Operator Precedence,” in *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, ACM, 1973, pp. 41–51.
- [15] Microsoft, “Roslyn: The .NET Compiler Platform.” 2015.
- [16] P. Dybjer, “Inductive Families,” *Formal Aspects of Computing*, vol. 6, pp. 440–465, 1994.

- [17] P. Wadler, “Theorems for Free!,” in *Proceedings of the 4th International Conference on Functional Programming Languages and Computer Architecture (FPCA)*, ACM, 1989, pp. 347–359.
- [18] J. Voigtländer, “Free Theorems Involving Type Constructor Classes,” in *Proceedings of the 14th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, ACM, 2009, pp. 173–184.
- [19] A. Mokhov, G. Lukyanov, S. Marlow, and J. Dimino, “Selective Applicative Functors,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. ICFP, 2019.

Appendices

Appendix A

Project Proposal

Query-based Incremental Elaborator for a Dependently Typed Language

A.0.1 Introduction and Description

This project aims to implement an incremental, query-based elaborator for a dependently typed toy language, for efficient type checking in interactive development environments, such as proof assistants. Traditional type checkers for dependent types operate in batch mode, re-checking entire programs after each change. This project will investigate query-based compilation techniques, building on Olle Fredriksson’s work and modern language servers, to enable efficient incremental type checking.

The core objective is to build an elaborator that can efficiently respond to local changes in source code by recomputing only the affected parts of the type checking process. This will be achieved using a query-based architecture where type checking operations are decomposed into memoisable queries with explicit dependencies. The implementation will use the Salsa framework, written in Rust, which provides the tools for incremental on-demand computation by automatically tracking dependencies and memoisation.

A.0.2 Starting Point

No prior work was done before October.

A.0.3 Substance and Structure

1. Core Language: a minimal dependently typed calculus with Π , universes, and basic inductive types. The syntax will support function definitions, pattern matching, and type annotations.
2. Query-based elaborator: bidirectional type checking decomposed into queries (e.g., `infer_type`, `check_type`, `normalise_term`). Each query can be memoised, with proper dependency tracking to enable incremental recomputation.
3. Incremental infrastructure: use of the Salsa framework to handle source file changes, manage parse trees, and dependency graphs for queries. Detection of changes and lazy memoisation strategies. This is the most important component of the project.

4. Performance evaluation framework: benchmarking suite to measure incremental performance against batch re-elaboration, including synthetic workloads simulating typical editing patterns.

Key algorithms include type unification, bidirectional type checking with constraint generation, normalisation for dependent types.

A.0.4 Success Criteria and Evaluation Plan

A.0.4.1 Success Criteria

1. Correctly type check a language with dependent types, including Pi types, Sigma types and inductive types
2. Demonstrate measurable performance improvement for incremental changes vs full re-elaboration

A.0.4.2 Evaluation Plan

- Correctness: test suite of programs including simple proofs of equality, existential and universal quantification (dependent pair and arrow types).
- Performance and scalability: Benchmarks measuring re-elaboration time for:
 - local definition changes
 - type signature modifications
 - cascading dependency updates

on sample programs, generated by a computer, of at least 1000 lines of code

A.0.5 Work Plan

A.0.5.1 Michaelmas Term

- 9 Oct – 22 Oct: Read Salsa documentation, study dependent type implementation, implement toy examples, design language syntax and semantics
- 23 Oct – 6 Nov: Implement elaborator for dependent types over a query-based interface
- 7 Nov – 19 Nov: Complete elaborator for dependent types, including identity types, dependent pairs; write example programs
- 20 Nov – 3 Dec: Decompose elaborator, and other stages of compilation process into fine-grained queries

A.0.5.2 Lent Term

- 22 Jan – 4 Feb: Implement incrementalisation with change detection, cache invalidation, and dependency tracking, utilising Salsa
- 5 Feb – 18 Feb: Complete incrementalisation, with source-based tracking
- 19 Feb – 4 Mar: Optimise incrementalisation, query granularity and dependency tracking
- 5 Mar – 18 Mar: Evaluation and performance benchmarks, with synthetic programs

A.0.5.3 Easter Term

- 30 Apr – 15 May: Dissertation

A.0.6 Resources Declaration

- Personal laptop (32 GB RAM, Intel Ultra 7 155) for development

- Backup strategy: Git repository on GitHub
- Contingency plan: Backup laptop available

A.0.7 Ethics Approval

Not required as no human participants or personal data involved.